

Task 4: The Full Story Consistency Checker

What You're Building

You've spent Tasks 1–3 learning the theory and building the engine. Now you wire it all together into a working claim verification system, measure how accurate it is, and then make it smarter using a better prompting technique.

By the end of this task, you'll have a system that can read a story, accept a claim like "Elara grew up with wealthy parents", and correctly output 0 (Inconsistent).

Week 4 Breakdown

Part 1 : The MVP

Concept: Retrieve relevant chunks → Feed to LLM → Get a verdict.

This is the moment everything from Tasks 1–3 comes together. The full pipeline is:

[Claim] → [Embed] → [Search Vector DB] → [Top 3 Chunks] → [Prompt LLM] → [Verdict: 0 or 1]

Your Task:

- Rebuild your Task 3 pipeline (chunking + vector DB) in a clean setup cell
- Build a `verify_claim(claim)` function that runs the full pipeline
- Test it manually on at least one consistent claim and one inconsistent claim

Deliverable: A Python function that correctly verifies a claim from your dataset.

Part 2 - Evaluation

Concept: You can't improve what you don't measure.

Running your MVP on one or two examples feels good, but it doesn't tell you how reliable the system really is. Now you build a proper test dataset and measure baseline accuracy.

Your Task:

- Create an evaluation dataset of at least 12 test cases. Include:
 - Easy consistent claims (direct facts stated in the story)

- Easy inconsistent claims (clear contradictions)
- Hard cases (subtle contradictions, paraphrased facts, negation, numbers)
- Run the dataset through your pipeline and record which cases pass and fail
- Produce an accuracy report broken down by case type

Deliverable: An accuracy report. Identify why it failed in specific cases - was it a retrieval failure (wrong chunks retrieved) or a reasoning failure (right chunks, wrong conclusion)?

Part 3 - Prompt Engineering: Chain-of-Thought

Concept: Standard prompts fail on implicit details. Forcing the model to show its work fixes this.

A standard prompt asks: "Is this consistent? Answer 1 or 0."

A Chain-of-Thought (CoT) prompt asks: "Step 1: What does the story say? Step 2: What does the claim say? Step 3: Do they match? Step 4: Final verdict."

The model has to read more carefully when it has to justify its answer.

Your Task:

- Implement a `build_cot_prompt()` function with explicit reasoning steps
- Re-run your evaluation dataset using the CoT prompt
- Compare CoT vs. standard accuracy side by side

Deliverable: Show one concrete example where the standard prompt gave the wrong answer, but the CoT prompt got it right. Explain in your own words why the step-by-step approach helped.

Colab Notebook where you will be implementing these tasks -

https://colab.research.google.com/drive/1c8feGCaigd6CYG74DD2xf_hxCbMtrIcl?usp=sharing

(make a copy of this notebook)

Dataset -

Books -

https://drive.google.com/drive/folders/1aZUgJc387xlFkUpCr_qxYMd5ewX3miVs?usp=sharing

Backstory (Train and Test) -

https://drive.google.com/drive/folders/1Cfq3O-gHASl8OI4qlfBj_ko6t1PhPObE?usp=sharing

Dataset Background

To help you understand and build the full pipeline correctly, here's how the dataset is structured.

1. Drive Folder Structure

The drive folder contains:

- **2 complete books** (raw story text files)
- A folder named **Backstory**, which contains:
 - `train.csv`
 - `test.csv`

2. Backstory Folder

The **Backstory** folder contains the structured ground-truth data used for training and evaluation.

Files:

- `train.csv`
- `test.csv`

These CSV files are essential for:

- Understanding how claims are labeled
- Building your evaluation dataset
- Measuring model accuracy
- Debugging retrieval vs reasoning failures

3. CSV File Structure

When you open `train.csv`, you will see the following columns:

- `book_name` — Name of the book the claim belongs to
- `char` — Character involved in the claim
- `caption` — Short description or summary of the claim
- `content` — The actual claim sentence
- `label` — Ground truth label
 - 1 → Consistent with the story
 - 0 → Inconsistent with the story

4. How This Connects to Your Pipeline

This dataset serves as the **ground truth reference** for your claim verification system.

You will:

1. Use the **books** to build your vector database (chunk → embed → store).
2. Use the **content column** as input claims.
3. Compare your model's predicted verdict with the **label column**.
4. Measure accuracy and analyze errors.

This structure allows you to clearly separate:

- Retrieval errors (wrong chunks retrieved)
- Reasoning errors (correct chunks retrieved, wrong conclusion)

This dataset is the backbone of your evaluation process and is critical for properly validating your Story Consistency Checker.

Resources

RAG & Pipelines:

- LangChain: Chat with Your Data
<https://www.deeplearning.ai/short-courses/langchain-chat-with-your-data/>
- RAG for LLMs — Prompting Guide
<https://www.promptingguide.ai/techniques/rag>
- Original RAG Paper
<https://arxiv.org/abs/2005.11401>

Evaluation:

- RAGAS: Evaluation Framework for RAG
<https://docs.ragas.io/en/latest/>
- Think about: precision, recall, and accuracy as metrics

Chain-of-Thought:

- Chain-of-Thought Prompting Paper
<https://arxiv.org/abs/2201.11903>
- Zero-Shot CoT Guide — Prompting Guide
<https://www.promptingguide.ai/techniques/cot>

Notes

- You are free to use any LLM: OpenAI, Anthropic, or a free local model via HuggingFace (all options are in the notebook)

- The sample story in the notebook (about a character named Elara) is provided for testing
- use your own story for richer results
- Your evaluation dataset quality matters a lot. Weak test cases will give you misleadingly high accuracy